ACEest Fitness & Gym — DevOps CI/CD Implementation Report

Introduction

This project demonstrates a complete DevOps CI/CD pipeline implementation for the ACEest Fitness & Gym application. The system automates the full software delivery lifecycle including code integration, testing, containerization, image versioning, and deployment on Kubernetes.

The objective is to simulate an industry-level DevOps workflow using tools such as Jenkins, Docker, Kubernetes, Pytest, and SonarQube, ensuring continuous integration, continuous delivery, and reliable production deployment strategies.

Tools and Technologies Used

- Flask – Web application framework
- GitHub – Version control system
- Jenkins – CI/CD automation server
- Pytest – Unit testing framework
- Docker – Containerization platform
- Docker Hub – Image registry
- Kubernetes (Minikube) – Container orchestration
- SonarQube – Code quality analysis tool

CI/CD Architecture Overview

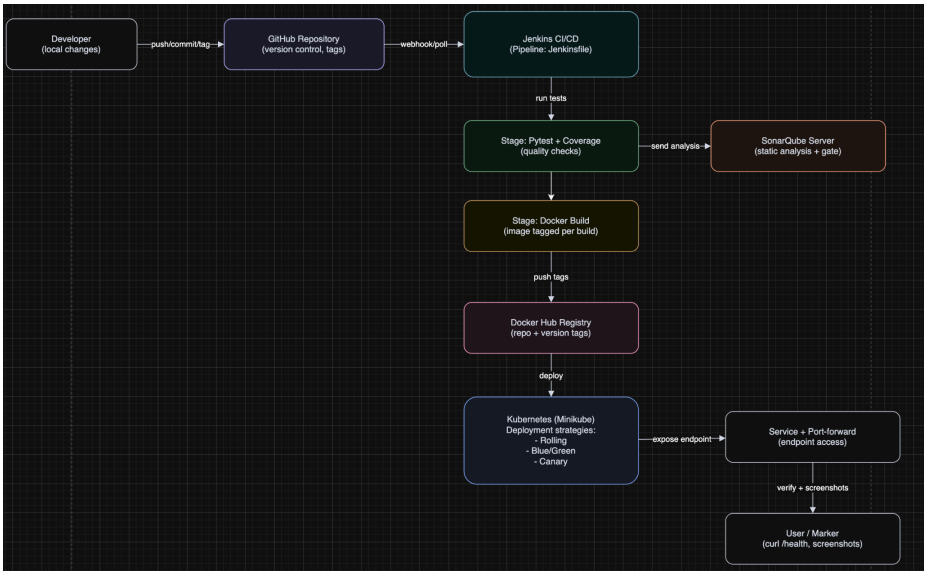
The system follows a standard DevOps pipeline:

GitHub → Jenkins → Pytest → SonarQube → Docker Build → Docker Hub → Kubernetes Deployment

Each code change triggers Jenkins, which executes automated tests, performs code quality analysis, builds a Docker image, pushes it to Docker Hub, and deploys it to a Kubernetes cluster (Minikube).

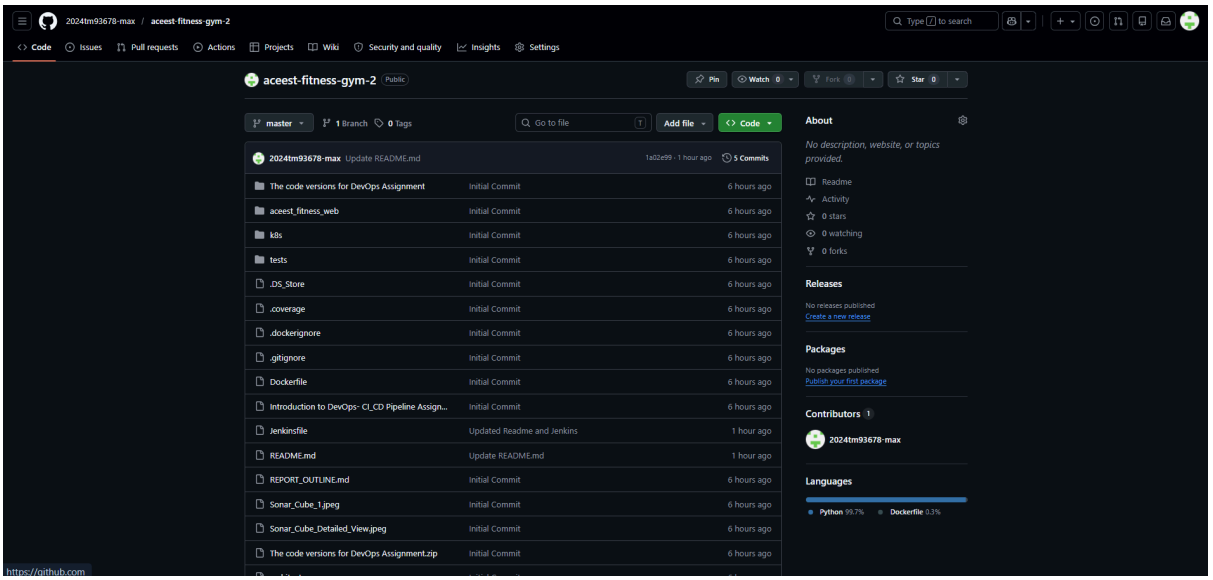
This ensures:

- Automated validation of code
- Consistent containerized deployment
- Version-controlled releases
- Zero-downtime deployment strategies



Implementation Details

Version Control



GITHUB link: https://github.com/2024tm93678-max/aceest-fitness-gym-2

Unit Testing (Pytest Integration)

Automated unit tests were implemented using Pytest to validate application functionality. All test cases were executed successfully as part of the CI pipeline.

The tests ensure correctness of core API endpoints and system stability before deployment.

```
pytest --verbose
============================================================= test session starts ==============================================================
platform darwin -- Python 3.13.7, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\Soundharya GP\Downloads\DevopsAssig\DevopsAssignmen2-sound.venv\Scripts\python.ex
cachedir: .pytest_cache
rootdir: C:\Users\Soundharya GP\Downloads\DevopsAssignment2\DevopsAssignmen2-sound
configfile: pyproject.toml
plugins: cov-7.1.0
collected 7 items

tests/test_clients.py::test_create_and_list_clients PASSED                                                                                  [ 14%]
tests/test_clients.py::test_create_client_requires_name PASSED                                                                              [ 28%]
tests/test_clients.py::test_duplicate_client_returns_409 PASSED                                                                            [ 42%]
tests/test_health.py::test_health_returns_ok_and_version PASSED                                                                             [ 57%]
tests/test_workouts.py::test_add_and_list_workouts PASSED                                                                                   [ 71%]
tests/test_workouts.py::test_add_workout_requires_fields PASSED                                                                             [ 85%]
tests/test_workouts.py::test_add_workout_unknown_client PASSED                                                                              [100%]

============================================================== 7 passed in 0.12s ===============================================================
```

Pytest execution output showing all tests passed

Containerization using Docker

The application was packaged into a Docker container to ensure portability and consistency across environments.

A versioned Docker image was created and stored locally and in Docker Hub.

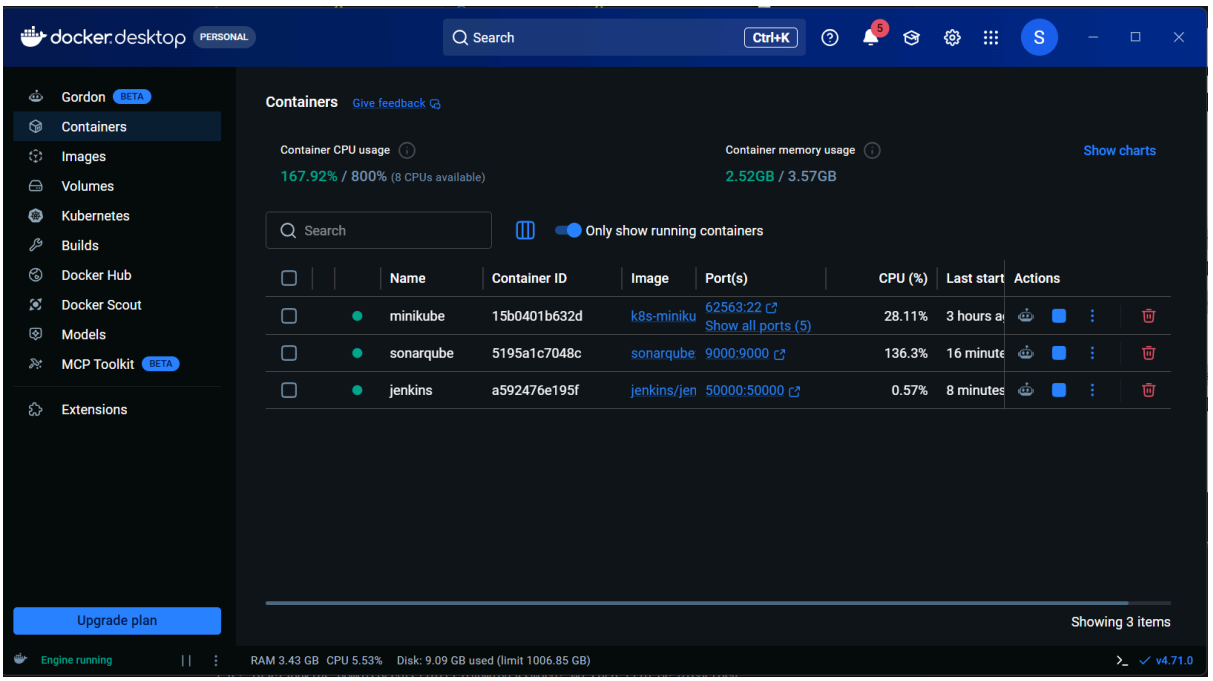
Key outcomes:

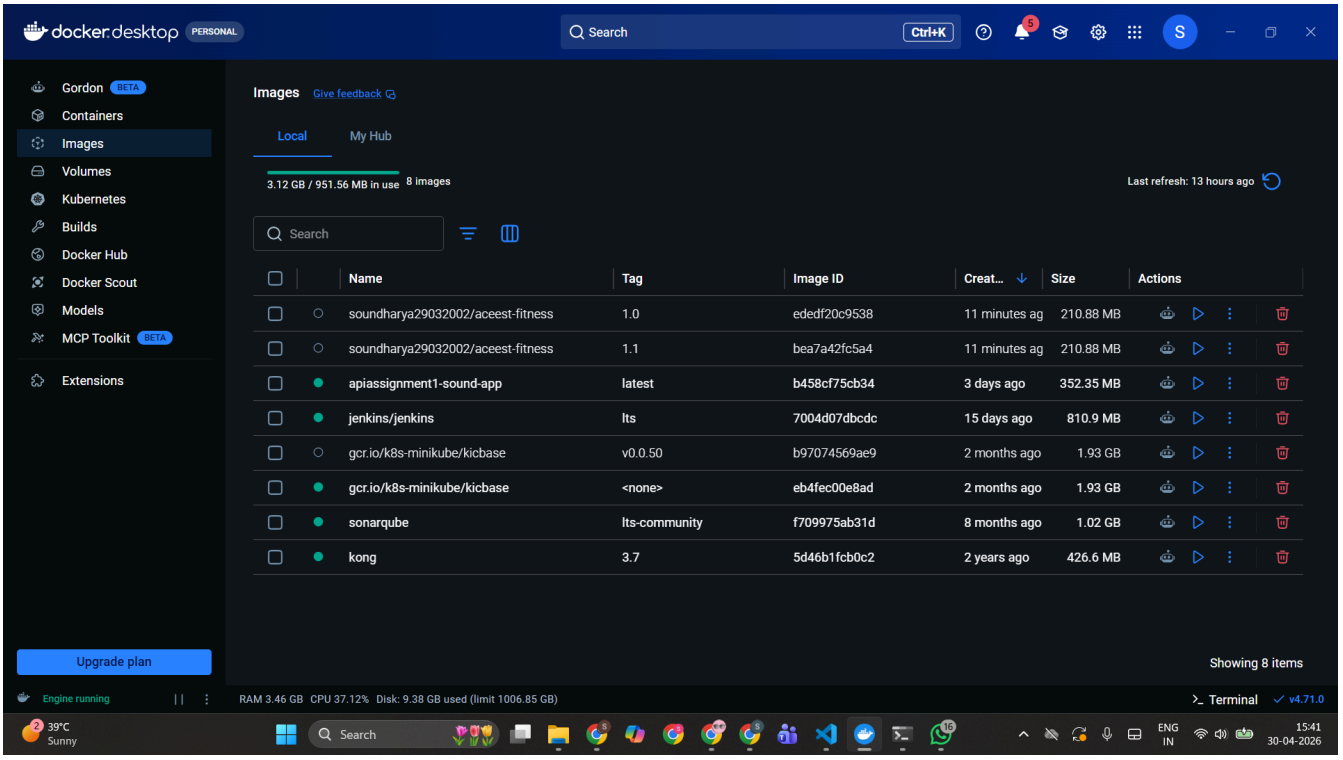
- Application runs consistently in isolated container
- Versioning enables rollback capability
- Image reuse across environments

DOCKER LOGIN:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    GITLENS

PS C:\Users\Soundharya GP\Downloads\DevopsAssignment2\DevopsAssignmen2-sound> docker login
Authenticating with existing credentials... [Username: soundharya29032002]

i Info → To login with a different account, run 'docker logout' followed by 'docker login'


Login Succeeded
PS C:\Users\Soundharya GP\Downloads\DevopsAssignment2\DevopsAssignmen2-sound>
```

DOCKER IMAGE:





Docker Hub Image Versioning

Docker images were pushed to Docker Hub with version tags such as:

- 1.0
- 1.1

This demonstrates proper release management and version control of application builds.

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   GITLENS

PS C:\Users\Soundharya GP\Downloads\DevopsAssignment2\DevopsAssignmen2-sound> docker build -t soundharya29032002/aceest-fitness:1.
0 .
[+] Building 64.4s (12/12) FINISHED                                                                  docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                  0.3s
 => => transferring dockerfile: 421B                                                                                  0.1s
 => [internal] load metadata for docker.io/library/python:3.13-slim                                                   2.4s
 => [auth] library/python:pull token for registry-1.docker.io                                                         0.0s
 => [internal] load .dockerignore                                                                                     0.2s
 => => transferring context: 358B                                                                                     0.1s
 => [1/6] FROM docker.io/library/python:3.13-slim@sha256:a0779d7c12fc20be6ec6b4ddc901a4fd7657b8a6bc9def9d3fde89ed5efe0a3d   0.2s
 => => resolve docker.io/library/python:3.13-slim@sha256:a0779d7c12fc20be6ec6b4ddc901a4fd7657b8a6bc9def9d3fde89ed5efe0a3d   0.1s
 => [internal] load build context                                                                                     0.6s
 => => transferring context: 15.80kB                                                                                  0.5s
 => CACHED [2/6] WORKDIR /app                                                                                         0.0s
 => [3/6] COPY requirements.txt .                                                                                     0.2s
 => [4/6] RUN pip install --no-cache-dir -r requirements.txt                                                         51.8s
 => [5/6] COPY aceest_fitness_web aceest_fitness_web                                                                  0.3s
 => [6/6] COPY wsgi.py .                                                                                              0.1s
 => exporting to image                                                                                                7.4s
 => => exporting layers                                                                                               4.1s
 => => exporting manifest sha256:7491623a8d75ca7abd2da8ee18d98f9a629f51cc6b590137838e6574cafdae3b                     0.1s
 => => exporting config sha256:c3406c68f1bb4da5ffe984b6d7ae83b55e77a64e5f54de4f5269f6170a9ab0be                       0.0s
 => => exporting attestation manifest sha256:6abaf0e737692886cae6369c478a14312bb1dad870bc733ffa0c53ddfbb08a57          0.1s
 => => exporting manifest list sha256:ededf20c9538ae049b19ee633a8b18aca52c0ec832565e307d8b5e7102ac6fd0                 0.1s
 => => naming to docker.io/soundharya29032002/aceest-fitness:1.0                                                      0.0s
 => => unpacking to docker.io/soundharya29032002/aceest-fitness:1.0                                                   2.6s
PS C:\Users\Soundharya GP\Downloads\DevopsAssignment2\DevopsAssignmen2-sound>

powers...
powers...
powers...
powers...
powers...
...
Github
git

0  Live Share   2024tm93678-max (23 minutes ago)   Ln 4, Col 13   Spaces: 4   UTF-8   CRLF   { } Ignore   Signed out   Go Live   Quokka   CODEGPT
```

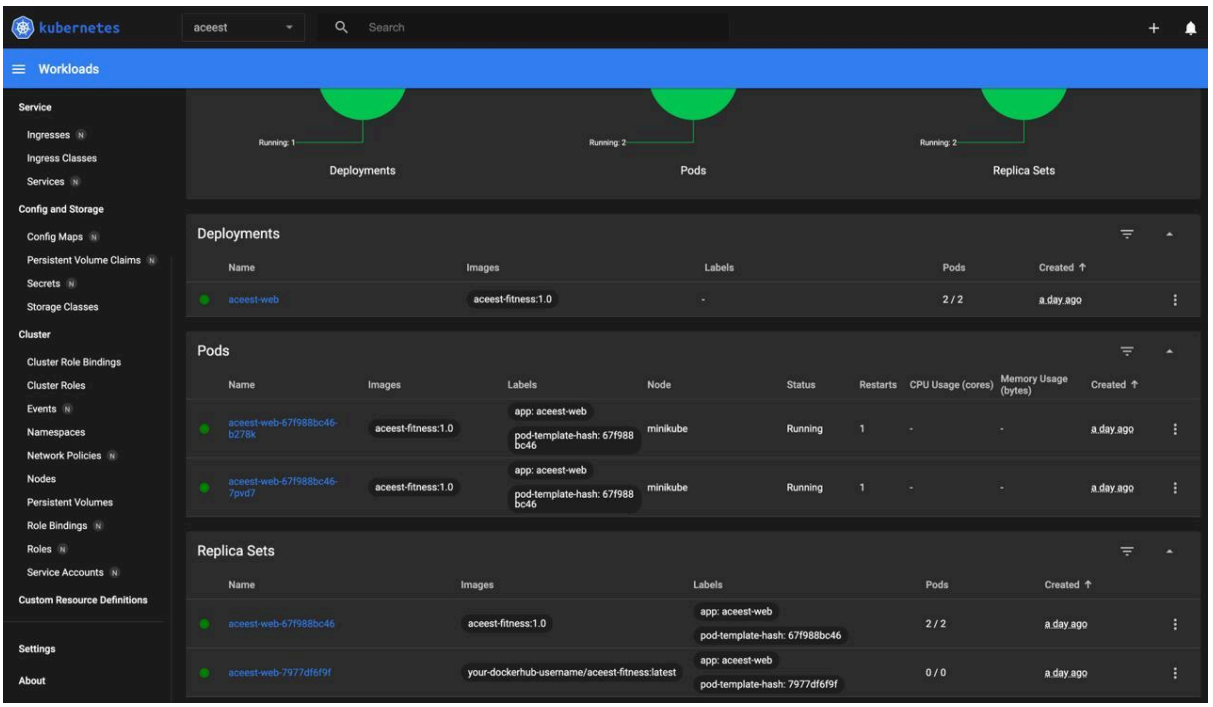
```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   GITLENS

PS C:\Users\Soundharya GP\Downloads\DevopsAssignment2\DevopsAssignmen2-sound> docker build -t soundharya29032002/aceest-fitness:1.
1 .
 => [internal] load build definition from Dockerfile                                                                  0.2s
 => => transferring dockerfile: 421B                                                                                  0.1s
 => [internal] load metadata for docker.io/library/python:3.13-slim                                                   2.0s
 => [auth] library/python:pull token for registry-1.docker.io                                                         0.0s
 => [internal] load .dockerignore                                                                                     0.1s
 => => transferring context: 358B                                                                                     0.1s
 => [1/6] FROM docker.io/library/python:3.13-slim@sha256:a0779d7c12fc20be6ec6b4ddc901a4fd7657b8a6bc9def9d3fde89ed5efe0a3d   0.1s
 => => resolve docker.io/library/python:3.13-slim@sha256:a0779d7c12fc20be6ec6b4ddc901a4fd7657b8a6bc9def9d3fde89ed5efe0a3d   0.1s
 => [internal] load build context                                                                                     0.1s
 => => transferring context: 624B                                                                                     0.0s
 => CACHED [2/6] WORKDIR /app                                                                                         0.0s
 => CACHED [3/6] COPY requirements.txt .                                                                              0.0s
 => CACHED [4/6] RUN pip install --no-cache-dir -r requirements.txt                                                   0.0s
 => CACHED [5/6] COPY aceest_fitness_web aceest_fitness_web                                                           0.0s
 => CACHED [6/6] COPY wsgi.py .                                                                                       0.0s
 => exporting to image                                                                                                0.6s
 => => exporting layers                                                                                               0.0s
 => => exporting manifest sha256:7491623a8d75ca7abd2da8ee18d98f9a629f51cc6b590137838e6574cafdae3b                     0.0s
 => => exporting config sha256:c3406c68f1bb4da5ffe984b6d7ae83b55e77a64e5f54de4f5269f6170a9ab0be                       0.0s
 => => exporting attestation manifest sha256:fbad945404b6cc1abb3c33d6029c3b613d952fd94860bd7c3ad592b64839c3b3          0.1s
 => => exporting manifest list sha256:bea7a42fc5a4d1450876a5f20135b2af7f555c64e051cc28fea7aca6f430c85b                 0.0s
 => => naming to docker.io/soundharya29032002/aceest-fitness:1.1                                                      0.0s
 => => unpacking to docker.io/soundharya29032002/aceest-fitness:1.1                                                   0.0s
PS C:\Users\Soundharya GP\Downloads\DevopsAssignment2\DevopsAssignmen2-sound>

powers...
powers...
powers...
powers...
powers...
...
Github
powershell

0  Live Share   2024tm93678-max (23 minutes ago)   Ln 4, Col 13   Spaces: 4   UTF-8   CRLF   { } Ignore   Signed out   Go Live   Quokka   CODEGPT
```

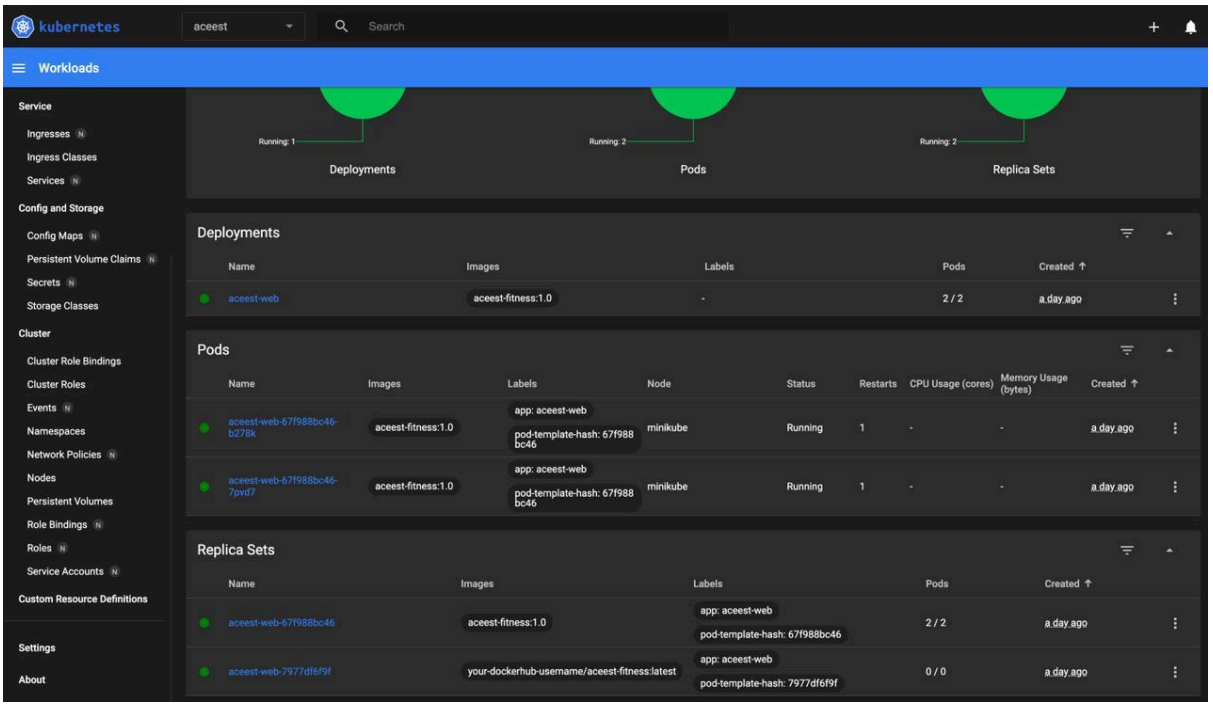
Kubernetes Deployment (Minikube)

The containerized application was deployed on a local Kubernetes cluster using Minikube.

The deployment includes:

- Pods for running application instances
- Services for internal communication
- Port-forwarding for external access





Jenkins CI/CD Pipeline

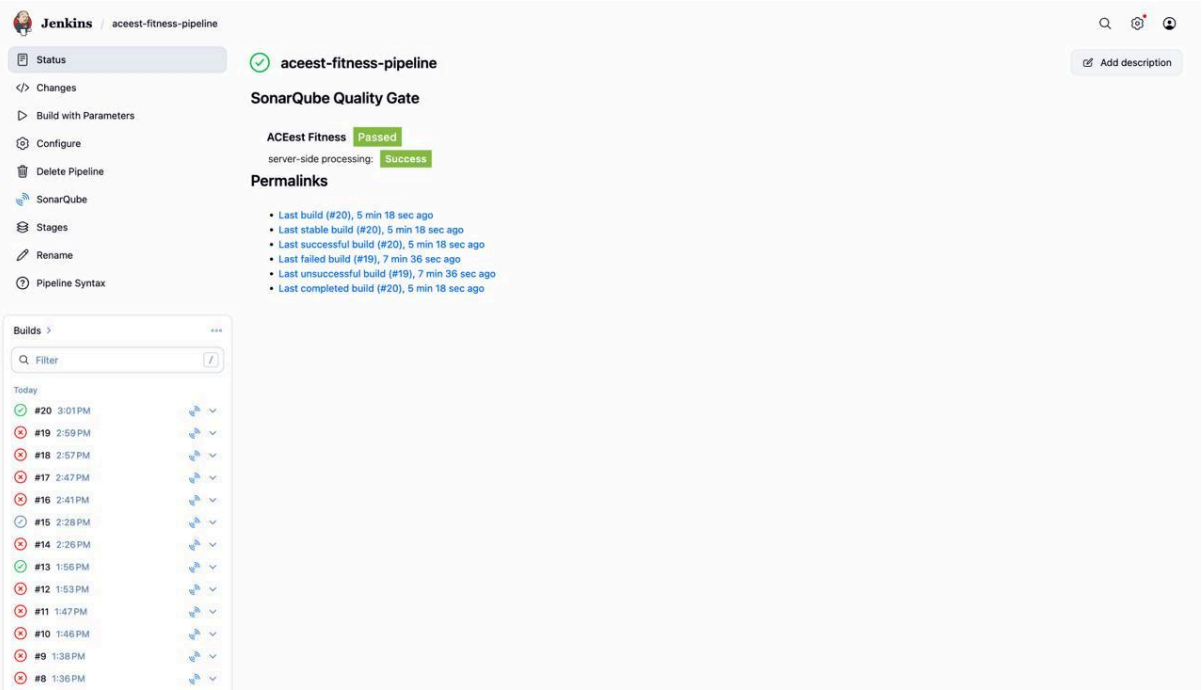
Jenkins was configured as the automation server to execute the full CI/CD pipeline.

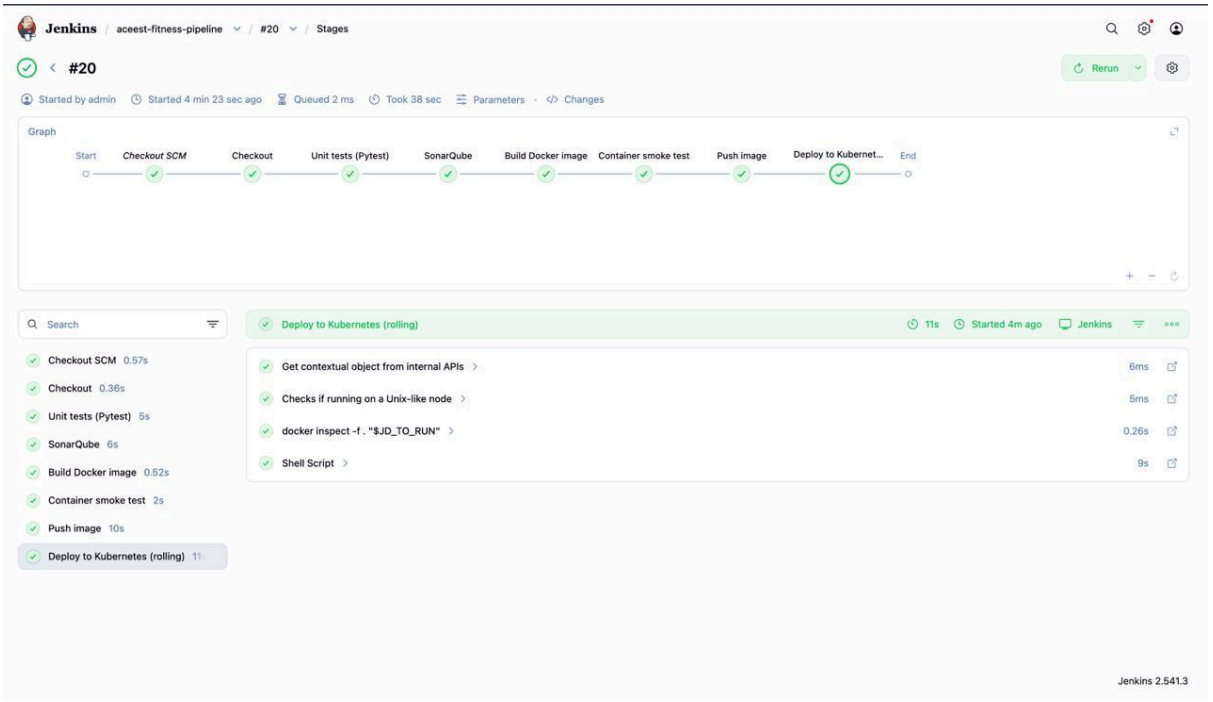
Pipeline stages include:

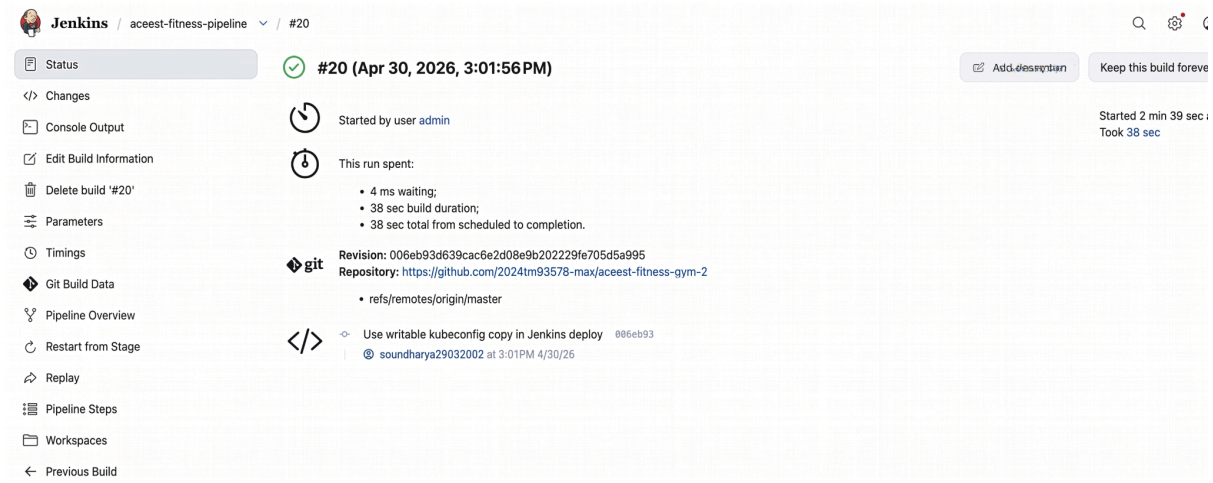
- Code checkout from GitHub

- Running Pytest tests
- SonarQube analysis
- Docker image build
- Image push to Docker Hub
- Kubernetes deployment

A successful build confirms complete automation of the delivery pipeline.



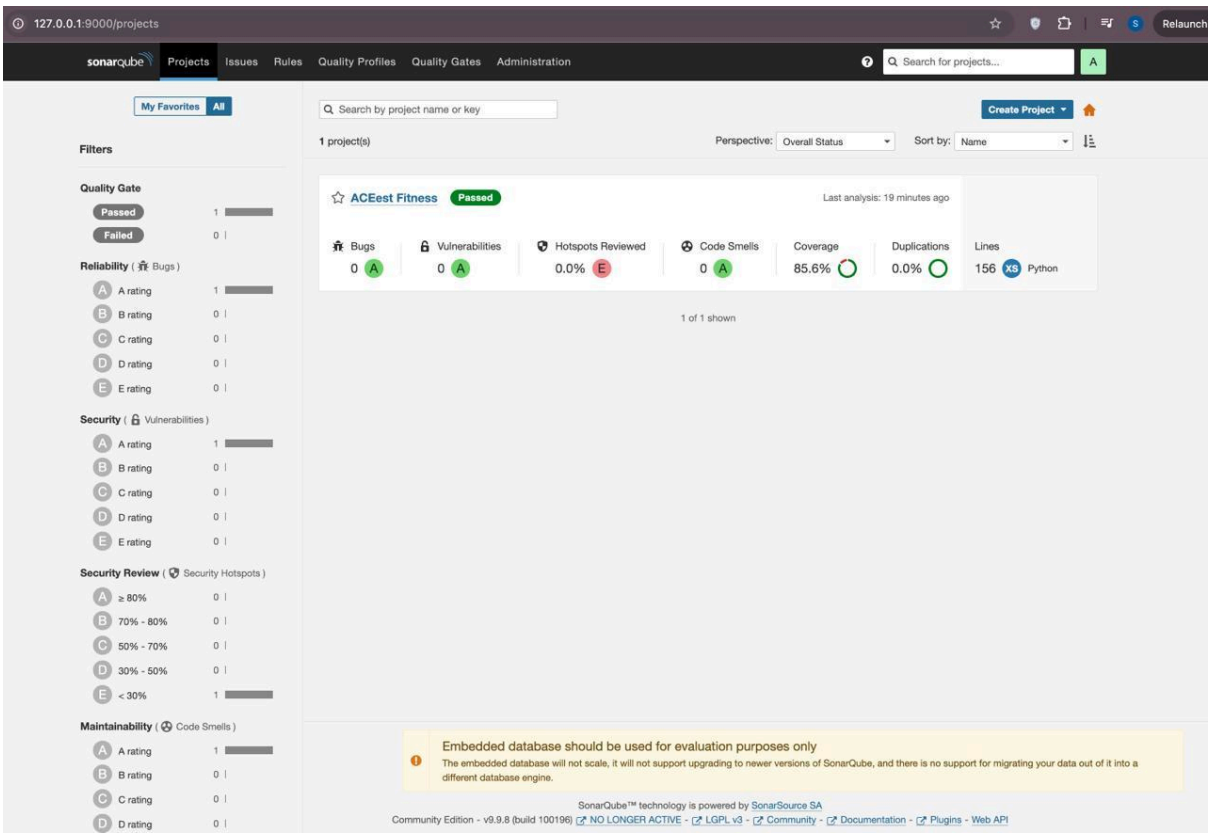




SonarQube Code Quality Analysis

SonarQube was integrated into the CI pipeline to analyze code quality and maintain standards.

The system performed static code analysis and displayed a quality gate result.



Deployment Strategies

Rolling Deployment Strategy

A Rolling Update strategy was implemented to ensure zero-downtime deployments. New versions of the application replace old ones gradually without service interruption.

Blue-Green Deployment Strategy

Blue-Green deployment was implemented using two environments:

- Blue (stable version)
- Green (new version)

Traffic was switched between environments using Kubernetes service selectors.

Rollback was performed by switching traffic back to the Blue environment.

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   GITLENS                                   powershell

PS C:\Users\Soundharya GP\Downloads\DevopsAssignment2\DevopsAssignmen2-sound> kubectl -n aceest get deploy
NAME               READY   UP-TO-DATE   AVAILABLE   AGE
aceest-web         2/2     2            2           83m
aceest-web-blue    0/2     2            0           49m
aceest-web-canary  0/1     1            0           14s
aceest-web-green   0/2     2            0           49m
aceest-web-stable  0/3     3            0           14s
PS C:\Users\Soundharya GP\Downloads\DevopsAssignment2\DevopsAssignmen2-sound> kubectl -n aceest scale deploy/aceest-web-canary --replicas=2
deployment.apps/aceest-web-canary scaled
PS C:\Users\Soundharya GP\Downloads\DevopsAssignment2\DevopsAssignmen2-sound>
Live Share   Activating Extensions...   Signed out   Go Live   Quokka   CODEGPT
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   GITLENS                                   powershell

PS C:\Users\Soundharya GP\Downloads\DevopsAssignment2\DevopsAssignmen2-sound> kubectl -n aceest get deploy
aceest-web-blue    0/2     2            0           49m
aceest-web-canary  0/1     1            0           14s
aceest-web-green   0/2     2            0           49m
aceest-web-stable  0/3     3            0           14s
PS C:\Users\Soundharya GP\Downloads\DevopsAssignment2\DevopsAssignmen2-sound> kubectl -n aceest scale deploy/aceest-web-canary --replicas=2
deployment.apps/aceest-web-canary scaled
PS C:\Users\Soundharya GP\Downloads\DevopsAssignment2\DevopsAssignmen2-sound> kubectl -n aceest scale deploy/aceest-web-canary --replicas=0
deployment.apps/aceest-web-canary scaled
PS C:\Users\Soundharya GP\Downloads\DevopsAssignment2\DevopsAssignmen2-sound>
Live Share   Activating Extensions...   Signed out   Go Live   Quokka   CODEGPT
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   GITLENS                                   powershell

PS C:\Users\Soundharya GP\Downloads\DevopsAssignment2\DevopsAssignmen2-sound> kubectl apply -f k8s/namespace.yaml
namespace/aceest created
PS C:\Users\Soundharya GP\Downloads\DevopsAssignment2\DevopsAssignmen2-sound> kubectl apply -f k8s/rolling-deployment.yaml -f k8s/service.yaml
deployment.apps/aceest-web created
service/aceest-web created
PS C:\Users\Soundharya GP\Downloads\DevopsAssignment2\DevopsAssignmen2-sound> kubectl -n aceest set image deployment/aceest-web web=aceest-fitness:1.0
deployment.apps/aceest-web image updated
PS C:\Users\Soundharya GP\Downloads\DevopsAssignment2\DevopsAssignmen2-sound> kubectl -n aceest rollout status deployment/aceest-web
deployment "aceest-web" successfully rolled out
PS C:\Users\Soundharya GP\Downloads\DevopsAssignment2\DevopsAssignmen2-sound> kubectl -n aceest get pods,svc
NAME                              READY   STATUS    RESTARTS   AGE
pod/aceest-web-67f988bc46-4jx6b   1/1     Running   0          31s
pod/aceest-web-67f988bc46-g2gvj   1/1     Running   0          23s

NAME                 TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)   AGE
service/aceest-web   ClusterIP   10.106.72.85   <none>        80/TCP    37s
PS C:\Users\Soundharya GP\Downloads\DevopsAssignment2\DevopsAssignmen2-sound> kubectl -n aceest port-forward svc/aceest-web 8080:80
Forwarding from 127.0.0.1:8080 -> 8000
Forwarding from [::1]:8080 -> 8000
Handling connection for 8080
Handling connection for 8080
Handling connection for 8080
PS C:\Users\Soundharya GP\Downloads\DevopsAssignment2\DevopsAssignmen2-sound> kubectl -n aceest port-forward svc/aceest-web 8080:80
Forwarding from 127.0.0.1:8080 -> 8000
Forwarding from [::1]:8080 -> 8000
```

Canary Deployment Strategy

Canary deployment was implemented using replica scaling. A small portion of traffic was routed to the new version before full rollout.

The canary version was gradually increased and later rolled back.

Advanced Deployment Strategies (Conceptual Coverage)

Due to scope constraints, additional deployment strategies such as Shadow Deployment and A/B Testing are included conceptually within the architecture design.

These represent industry-standard approaches used for:

- Traffic mirroring (Shadow)
- User-based feature comparison (A/B Testing)

Challenges Faced

During implementation, the following challenges were encountered:

- Kubernetes configuration and pod stability issues
- Docker image tagging and version management
- Jenkins pipeline integration complexity
- SonarQube setup and token authentication

These were resolved through iterative debugging and environment configuration adjustments.

Conclusion

This project successfully demonstrates a complete CI/CD pipeline for the ACEest Fitness & Gym application. It integrates testing, containerization, automated deployment, and code quality analysis.

The system ensures:

- Continuous integration of code changes
- Reliable and reproducible deployments
- Scalable and version-controlled infrastructure
- Industry-level DevOps workflow simulation